

Skip List – Introduction

Skip Lists

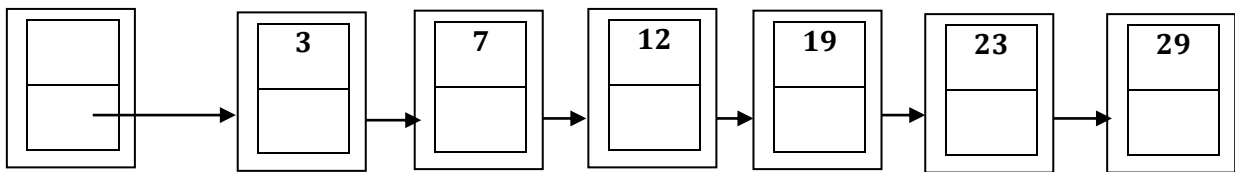
Binary trees can be used for representing abstract data types such as dictionaries and ordered lists. They work well when the elements are inserted in a random order. Some sequences of operations, such as inserting the elements in order, produce degenerate data structures that give very poor performance. If it were possible to randomly permute the list of items to be inserted, trees would work well with high probability for any input sequence. In most cases queries must be answered on – line, so randomly permuting the input is impractical. Balanced tree algorithms re – arrange the tree as operations are performed to maintain certain balance conditions and assure good performance.

Skip lists are a probabilistic alternative to balanced trees. Skip list is a data structure that can be used as an alternative to balanced binary trees . As compared to a binary tree, skip lists allow quick search, insertion and deletion of elements. This is achieved by using probabilistic balancing rather than strictly enforce balancing. It is basically a linked list with additional pointers such that intermediate nodes can be skipped. It uses a random number generator to make some decisions.

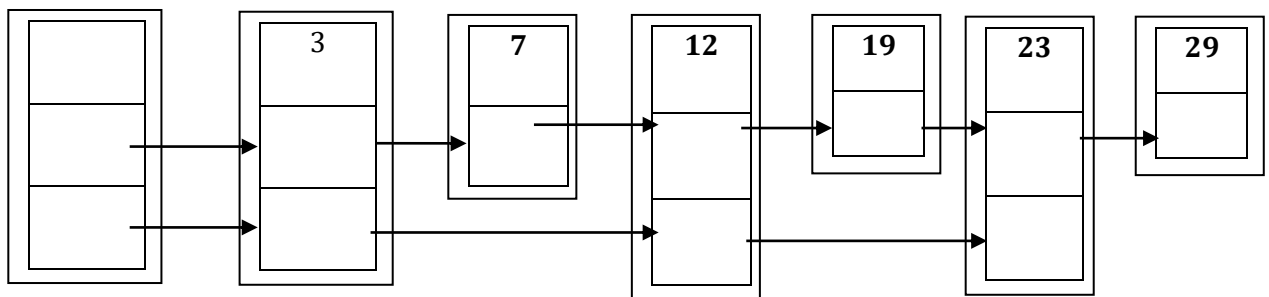
In an ordinary sorted linked list, search, insert, and delete are in $O(n)$ because the list must be scanned node – by – node from the head

to find the relevant node. If somehow we could scan down the list in bigger steps (skip down, as it were), we would reduce the cost of scanning. This is the fundamental idea behind Skip Lists.

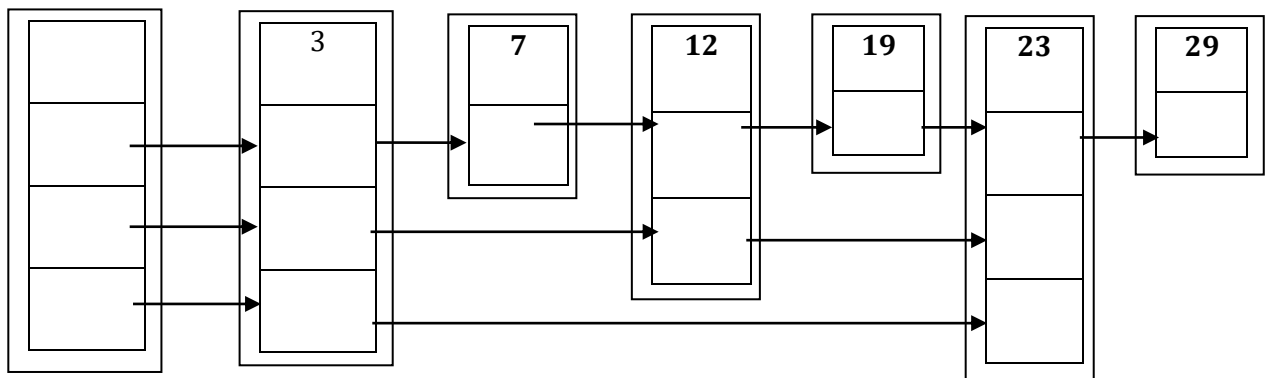
Skip Lists with One Level



Skip Lists with Two Level



Skip Lists with Three Level



Performance

In a simple linked list that consists of n elements, to perform a search n comparisons are required in the worst case. If a second pointer pointing two nodes ahead is added to every node, the number of comparisons goes down to $\frac{n}{2} + 1$ in the worst case.

Adding one more pointer to every fourth node and making them point to the fourth node ahead reduces the number of comparisons to $\left\lceil \frac{n}{4} \right\rceil + 2$. If this strategy is continued so that every node with i pointers points to 2^{i-1} nodes ahead, $O(\log n)$ performance is obtained and the number of pointers has only doubled $\left(n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \frac{n}{16} + \dots = 2n \right)$.

Level	Pointer spacing [No. of Pointers]	Number of nodes having that pointer
1	1	$\frac{n}{1}$
2	2	$\frac{n}{2}$
3	4	$\frac{n}{4}$
4	8	$\frac{n}{8}$
5	16	$\frac{n}{16}$
...	...	...

So the i – th level skips approximately:

$$\boxed{2^{i-1}}$$

nodes.

For example:

$$i = 1 \Rightarrow 2^0 = 1$$

$$i = 2 \Rightarrow 2^1 = 2$$

$$i = 3 \Rightarrow 2^2 = 4$$

$$i = 4 \Rightarrow 2^3 = 8$$

and so on.

Suppose there are $n = 16$ elements.

The levels can jump:

$$1, 2, 4, 8, 16$$

The number of levels needed is approximately:

$$\log_2 n$$

because,

$$2^k \approx n$$

gives,

$$k \approx \log_2 n \left[\begin{array}{l} \text{By Logarithmic} \\ \text{Definitions} \\ \text{if } a^n = b, \text{ then:} \\ \log_a b = n \end{array} \right]$$

Therefore, the search can be performed in approximately

$$\boxed{O(\log n)}$$

expected time.

At each level, fewer nodes need the extra pointer:

$$n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \frac{n}{16} + \dots$$

Factor out n:

$$n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots \right)$$

The expression inside the parentheses is an infinite geometric series:

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 2$$

Therefore:

$$n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots = 2n$$

So the total number of forward – pointer slots is approximately 2n.

We have increased the pointer storage from approximately ' n ' pointers in an ordinary linked list to approximately ' $2n$ ' pointers in the skip – list structure. Yet the search performance improves from ' $O(n)$ ' in an ordinary linked list to ' $O(\log n)$ ' in the skip – list.

In simple terms, Skip Lists are sorted linked lists with two differences:

- *The nodes in an ordinary list have one next reference. The nodes in a Skip List have many next references (also called forward references).*
- *The number of forward references for a given node is determined probabilistically.*

We speak of a Skip List node having levels, one level per forward reference.

The number of levels in a node is called the size of the node.

*In an ordinary sorted list, **insert**, **remove**, and **find** operations require sequential traversal of the list. This results in $O(n)$ performance per operation. Skip Lists allow intermediate nodes in the list to be skipped during a traversal – resulting in an expected performance of $O(\log n)$ per operation.*
